

GENERICS

// class  
class Box<T> { }  
// variable  
Box<String> b = new Box<String>();  
// method  
public <T> void doStuff(T value) { }

JAVA BEING STUPID

T obj = new T(); // error  
obj instanceof T; // error  
T heh = (T) "asdf"; // no typechecking  
var s = Stack<int>(); // no primitive types  
private static T x; // static no worky  
Node<Number> s = new Node<Integer>; // error  
class IThrowAnErrorBecauseWhyNot {  
 void m(List<String> list) { }  
 void m(List<Integer> list) { }  
} // error because of type erasure & identifiability  
Integer[] c = set.toArray(new Integer[0]);

ITERATOR

for (String s : stringList) { // ==  
 for (Iterator<String> i = stringList.iterator();  
 i.hasNext(); ) {  
 String s = i.next();  
 }  
}

ITERABLE

interface Iterable<T> {  
 Iterator<T> iterator();  
}  
  
interface Iterator<E> {  
 boolean hasNext();  
 E next();  
}

COMPARABLE

TODO:  
Type signature?

static <T extends Comparable> T doStuff(T a, T b) {  
 // compareTo is possible with all types  
 return a.compareTo("b") > 0 ? a : b;  
}  
  
static <T extends Comparable<T>> T doStuff(T a, T b)  
{  
 // compareTo is possible only with T  
 return a.compareTo(b) > 0 ? a : b;  
}

EXAMPLE

class Graphic {  
 public void draw() { }  
}  
  
class Rectangle extends Graphic { }  
class Circle extends Graphic { }  
class GraphicStack<T extends Graphic>  
 extends Stack<T> {  
 public void drawAll() {  
 for (T item : this) item.draw();  
 }  
}

BYTECODE

TODO:

TYPE ERASURE

- Introduced because of "bAcKwArDs CoMpAtAbIlItY"  
- No type information at runtime (Non-Reifiable Type)

class MyStack<T> {  
 Entry<T> top;  
 int push(T value) { }  
}  
// becomes  
class MyStack {  
 Entry top;  
 int push(Object value) { }  
}  
/\* with bounds \*/  
class MyStack<T extends Comparable<T>> {  
 Entry<T> top;  
 int push(T value) { }  
}  
// becomes

class MyStack {  
 Entry top;  
 int push(Comparable value) { }  
}  
/\* what happens at/before runtime \*/  
MyStack<String> s = new MyStack<String>();  
s.push("Eh");  
String x = s.pop();  
// becomes  
MyStack s = new MyStack();  
s.push("Eh");  
String x = (String) s.pop();

WILDCARDS

void printGs(List<Graphic> gs) { }  
List<Graphic> gs1 = new ArrayList<>();  
printGs(gs1); // ok  
List<Circle> gs2 = new ArrayList<>();  
printGs(gs2); // error  
/\* solution \*/  
void printGs(List<? extends Graphic> gs) { }

GENERIC VARIANCE

|                | Type           | Compatible<br>Type-Args | R | W |
|----------------|----------------|-------------------------|---|---|
| Invariance     | C<T>           | T                       | ✓ | ✓ |
| Covariance     | C<? extends T> | T and sub-types         | ✓ | × |
| Contravariance | C<? super T>   | T and basis-types       | × | ✓ |
| Bivariance     | C<?>           | All                     | × | × |

INVARIANCE

static <T> void move(Stack<T>, from, Stack<T> to) {  
 while(!from.isEmpty()) to.push(from.pop());  
}  
  
var rectS = new Stack<Rectangle>();  
var graphS = new Stack<Graphic>();  
graphS.push(new Rectangle()); // ok  
move(rectS, graphS); // error  
Stack<Object> objS = graphS; // error  
/\* anotherone \*/  
String[] strA = new String[10];  
Object[] objA = strA; // ok  
objA[0] = Integer.valueOf(2); // runtime err

TODO:

// compiles?  
List<Graphic> list = new ArrayList<Rectangle>();  
// compiles?  
Rectangle[] rectangleStack = new Rectangle[10];  
Graphic[] graphicStack = new Graphic[10];  
graphicStack = rectangleStack;  
// compiles?  
Stack<Rectangle> rectangleStack = new Stack<>();  
Stack<Graphic> graphicStack = new Stack<>();  
graphicStack = rectangleStack;  
// compiles?  
Stack<Rectangle> rectangleStack = new Stack<>();  
Stack<Graphic> graphicStack = new Stack<>();  
graphicStack = rectangleStack;  
for (Rectangle r : rectangleStack) {  
 graphicStack.push(r);  
}

COVARIANCE

public class Stack<E> {  
 public void pushAll(Iterable<? extends E> src) { }  
}  
  
Stack<? extends Graphic> graphS;  
graphS = new Stack<Rectangle>(); // ok  
graphS = new Stack<Circle>(); // ok  
graphS = new Stack<Object>(); // error (duh)  
//  
/\* read only \*/  
Stack<? extends Graphic> stack = new  
 Stack<Rectangle>();  
stack.push(new Graphic()); // error  
stack.push(new Rectangle()); // error  
stack.push(new Triangle()); // error

CONTRAVARIANCE

static <T> void addToC(List<? super T> list, T e) {  
 list.add(e);  
}  
  
addToC(new ArrayList<Number>(), 3); // ok  
addToC(new ArrayList<Object>(), 3); // ok

/\* shenanigans \*/  
Stack<? super Graphic> s = new Stack<Object>();  
s.add(new Graphic()); // ok  
s.add(new Rectangle()); // ok  
Graphic g = s.pop(); // error  
Object g = s.pop(); // ok

BIVARIANCE

static void printList(List<?> list) {  
 for (Object elem : list)  
 System.out.print(elem + " ");  
}  
/\* more shenanigans \*/  
static void appendNewObject(List<?> list) {  
 list.add(new Object()); // error  
}

PRODUCER / CONSUMER

TODO:

MISC

<T extends Comparable & Collection> // multiple type  
 bounds

GENERICS STREAM TOMFOOLERY

List<? extends Media> medialis;  
public <S extends T> List<S>  
 filterBySubtype(Class<S> subtype) {  
 return medialis.stream()  
 .filter(subtype::isInstance)  
 .map(subtype::cast)  
 .collect(Collectors.toList());  
 }  
}

ANNOTATIONS

- Metadata  
- Not actually part of the code

TODO:  
slides 10

- @Override  
- @Test  
- @Json...

DEFINING

@Target(ElementType.TYPE)  
@Retention(RetentionPolicy.RUNTIME)  
public @interface Author {  
 String name();  
 String date() default "N/A";  
}

@Author(name = "UwU", date = "idon't validate input")  
public class WeeDoo { }

REFLECTON

- Information of Class (private and public): Methods, Attrib-  
 utes, Parent class, Implemented interfaces  
- Calling methods possible  
- Usages: Debugger, Serialization, Frameworks, Remote  
 Procedure Call, ORM

// all of these `throws SecurityException`  
public Field[] getDeclaredFields()  
public Method[] getDeclaredMethods()  
public Constructor<?>[] getDeclaredConstructors()  
  
System.out.println(dump(new Student("a", "b"), 0));  
// {  
// firstName = a  
// lastName = b  
// }  
Class c = "s".getClass(); // java.lang.String

TODO:  
Diagram (slides 26)

METHOD

// @Target(ElementType.METHOD)

public String getName()  
public boolean isAnnotationPresent(  
 Class<? extends Annotation> annotationClass)  
public Object invoke(Object obj, Object... args)

throws IllegalAccessException,  
 IllegalArgumentException,  
 InvocationTargetException

public class Profiler {  
 public static void main(String[] args) {  
 var testFunctions = new ProfileFunctions();  
 int[] array = {5, 2, 8, 1, 93, 3, 33, 1, 333};  
 var methods = ProfileFunctions.class  
 .getDeclaredMethods();  
 for (var m : methods) {  
 if (m.isAnnotationPresent(Profile.class)) {  
 m.setAccessible(true); // best practice or  
 sth, idk, i write code in actually good languages  
 Profiler.profileMethod(testFunctions, m,  
 new Object[] {array});  
 }  
 }  
 }  
}

CLASS

TODO:

// @Target(ElementType.TYPE)

getDeclaredConstructor().newInstance()  
...  
  
ATTRIBUTE

TODO:

// @Target(ElementType.FIELD)

...  
  
VALIDATION

@Min(value = 18, message = "Age must be ≥ 18")  
@Max(value = 99, message = "Age must be ≤ 99")  
private int age;  
@NotNull(message = "Name cannot be null")  
private String name;

ALGORITHMS

- Brute-force  
- Greedy  
 - Take best option each step (optimize locally)  
 - Fast  
- Backtracking  
 - Trial and error  
 - Best overall option  
 - Higher compute costs  
- Divide and conquer  
 - Binary search  
- Dynamic programming  
 - Recursion optimization?  
 - Tabulation

BACKTRACKING

TODO:  
slides 87

BIG O NOTATION

- Worst case scenario  
- Atomic operations = constant time  
- Runtime measured as sum of primitive operations

Notation

$$\widetilde{f} \left( \underbrace{n}_{\text{Size of problem}} \right) = \text{Number of steps}$$

g = complexity class

TODO:  
diagram  $f \leq cg$

$f(n)$  is  $O(g(n))$  if  $c > 0 \wedge n_0 \geq 1$   
 $f(n) \leq cg(n)$  for  $n \geq n_0$

f in order g:

Let  $f, g : \mathbb{N} \rightarrow \mathbb{N}$   
 $n_0, c \in \mathbb{N} \wedge \forall n \geq n_0 : f(n) \leq c \cdot g(n)$   
 $\Rightarrow f \in O(g) \Leftrightarrow f = O(g)$

Big O

$$\overset{\text{Big O}}{\sim} \left( \underbrace{n}_{\text{Size of problem}} \right) = \text{Complexity class}$$

RULES

If  $f(n)$  is polynomial of degree  $d$ , then  $f(n) \in O(n^d)$   
- ignore lower powers  
- ignore constants

Use most optimal function (lowest power)  
-  $2n$  is  $O(n)$  better than  $2n$  is  $O(n^2)$

Simplify as far as possible  
-  $3n + 5$  is  $O(n)$  instead of  $3n + 5$  is  $O(3n)$

COMPLEXITY CLASSES

| Name        | Class      | Example                    |
|-------------|------------|----------------------------|
| Constant    | 1          | Array read                 |
| Logarithmic | $\log n$   | Binary search              |
| Linear      | $n$        | Linear search              |
| Log-linear  | $n \log n$ | Merge+Quick sort           |
| Quadratic   | $n^2$      | Bubble+Insertion sort      |
| Cubic       | $n^3$      | Solve equation             |
| Polynomial  | $n^k$      | Various algorithms         |
| Exponential | $2^n$      | Calculate all subsets      |
| Factorial   | $n!$       | Calculate all permutations |

$O(1)$   
 $O(\log n)$   
 $O(n)$   
 $O(n \log n)$   
 $O(n^2)$   
 $O(n^3)$   
 $O(2^n)$

TODO:  
Counting primitive operations (slides 55)

SORTING ALGORITHMS

SELECTION SORT

- Search for smallest elem in unsorted part and move in front

- Less mutations in array
- Same performance even if list almost sorted

```
public static void selectionSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int minimum = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minimum])
                minimum = j;
        swap(a, i, minimum);
    }
}
```

$$(n-1) + (n-2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$$

#### INSERTION SORT

- Take elem and put into correct place in sorted part
- Good for already partially sorted arrays

```
public static void insertionSort(int[] a) {
    int n = a.length;
    for (int i = 1; i < n; i++)
        for (int j = i; j > 0 && a[j] < a[j - 1]; j--)
            swap(a, j, j - 1);
}
```

$$1 + \dots + (n-1) + n = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \sim O(n^2)$$

#### BUBBLE SORT

- Compare neighboring elems and swap if needed

```
public static void bubbleSort(int[] a) {
    for (n = a.length; n > 1; n--)
        for (i = 0; i < n - 1; i++)
            if (a[i] > a[i + 1])
                swap(a, i, i + 1);
}
```

$$(n-1) + (n-2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$$

#### COUNTING SORT

```
public static void countingSort(int[] a) {
    int k = Arrays.stream(arr).max().getAsInt();
    int[] c = new int[k + 1];
    for (int i : a) c[i]++;
    int pos = 0;
    for (int i = 0; i < k; i++)
        for (int j = 0; j < c[i]; j++) {
            a[pos] = i;
            pos++;
        }
}
```

$$n + n + k + n = 3n + k \sim O(n + k)$$

#### SHELL SORT

- Sort pairs of far away elems
- Sort the partially sorted array through insertion sort

```
public static void shellSort(int[] a) {
    int n = a.length;
    int h = 1;
    while (h < n/3) h = 3 * h + 1;
    while (h >= 1) {
        for (int i = h; i < n; i++)
            for (int j = i; j >= h && a[j] < a[j - h]; j = j - h) swap(a, j, j - h);
        h /= 3;
    }
}
```

#### BINARY SEARCH

```
public static <T extends Comparable<T>> boolean bs(
    List<T> data, T target, int low, int high) {
    if (low > high) return false;
    int pivot = low + ((high - low) / 2);
    if (target.equals(data.get(pivot)))
        return true;
    else if (target.compareTo(data.get(pivot)) < 0)
        return searchBinary(data, target, low, pivot - 1);
    else
        return searchBinary(data, target, pivot + 1, high);
}
```

```
return searchBinary(data, target, pivot + 1, high);
}
```

$$O(\log n)$$

#### RECURSION

When a function calls itself

#### LINEAR RECURSION

Recursive call starts **at most one** further recursive call

#### TODO:

recursive → iterative (slides 61)

#### TAIL RECURSION

Linear recursion where the recursive call is **last**

#### TODO:

recursive → iterative (slides 69 – nice)

#### BINARY RECURSION

All non-terminal calls have **two** recursive calls

#### TODO:

example (slides 76,77,78)

#### TODO:

$O(n)$  (slides 84)

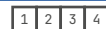
#### DATA STRUCTURES

ADT: Abstract Data Type (e.g. Interface)

#### TODO:

diagram

#### ARRAY



#### LINKED LIST

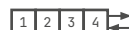


#### DOUBLY LINKED LIST



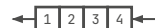
#### STACK

– LIFO

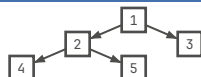


#### QUEUE

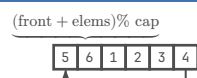
– FIFO



#### TREE



#### RING BUFFER



#### PRIORITY QUEUE

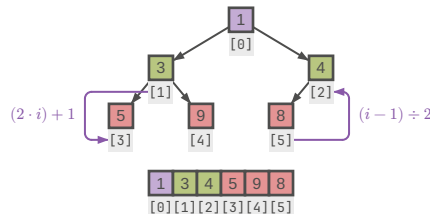
| Method    | Unsorted List | Sorted List |
|-----------|---------------|-------------|
| insert    | $O(1)$        | $O(n)$      |
| min       | $O(n)$        | $O(1)$      |
| removeMin | $O(n)$        | $O(1)$      |

#### HEAP

- Binary tree
- Layers  $0, \dots, h-1$  fully populated, layer  $h$  filled from left
- Operations always lowest RHS so that tree stays consistent

**Min-Heap:** Keys of nodes are **smaller** or equal than children's

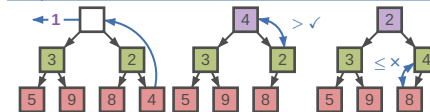
**Max-Heap:** Keys of nodes are **larger** or equal than children's



#### ENQUEUE



#### DEQUEUE



#### HEAP SORT

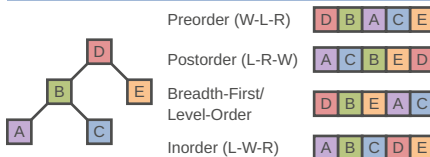
Dequeue elems from head into list, resorting tree on each iter

#### TODO:

#### BINARY SEARCH TREE (BST)

- All subnodes of **left** subtree are **smaller** than root
- All subnodes of **right** subtree are **larger** than root

#### TRAVERSING



#### TODO:

#### MORE DATA STRUCTURES

**Set:** Elements of same type, no duplicates, without order

**Multiset:** Set with duplicates

**Map:** KV-pairs with unique keys

**Multimap:** Key can have multiple values

#### HASHING

Hash-function  $h$  maps  $e$  onto  $[0, n-1]$ .  $h(e)$  = position of  $e$

$$e \xrightarrow{\text{hash code}} \mathbb{Z} \xrightarrow{\text{compression function}} [0, n-1]$$

Properties of good hash functions:

- Equal distribution
- Fast computation
- Constant complexity
- Deterministic
- Includes as much information from key as possible

#### HASH FUNCTIONS

- Modulo primes
- Memory address (Object.hashCode() default)
- Byte/Integer cast ByteBuffer.wrap(b).getInt()
- Component sum (eg. sum string char codepoints)
- Polynome accumulation  $a_0 + a_1z + a_2z^2 + \dots + a_{n-1}z^{n-1}$

#### ANOMALIES

- Empty spaces
- Collisions
- Overflow

#### MANAGING HASH COLLISIONS

**Closed addressing/Separate chaining:** Each bucket is a list. Search:  $O(\text{entries/buckets})$

**Open addressing:** "overflows" into next empty bucket

#### EQUALITY

It is generally necessary to override hashCode whenever equals is overridden (equal objects = equal hashCodes)